

1.0 - NEURAL NETWORKS INTRODUCTION

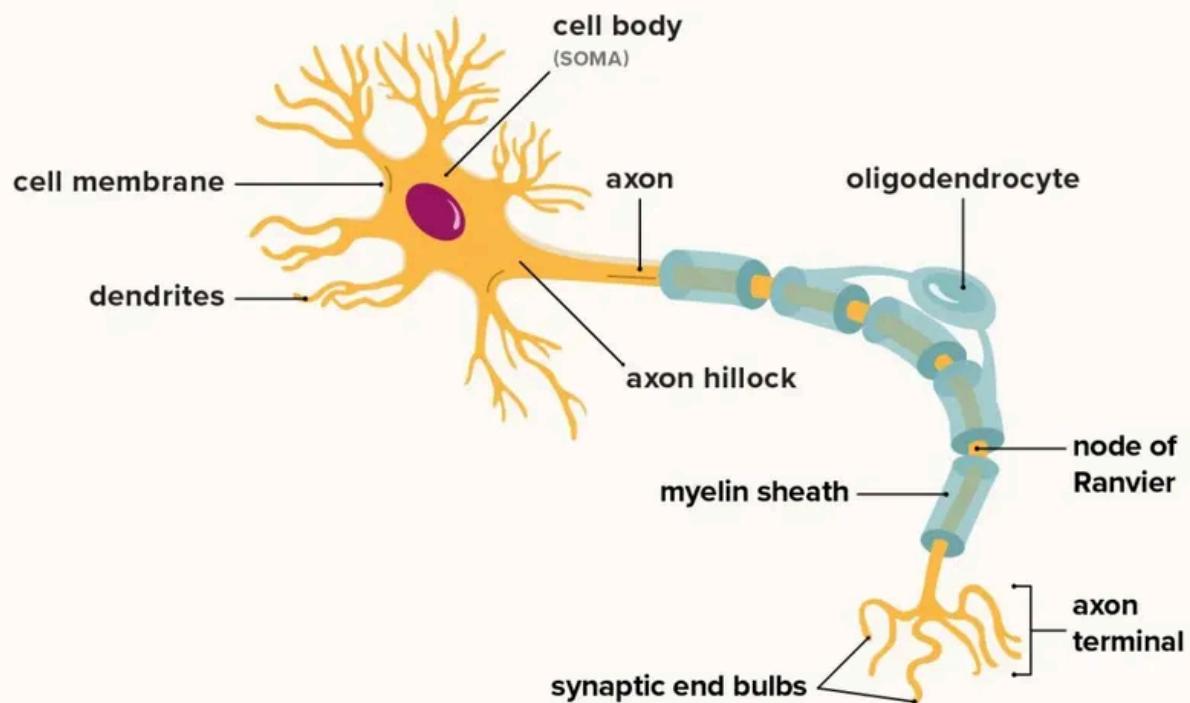
CHAPTER INDEX

- ☐ [NEURONS](#)
 - [MC CULLOCH - PITTS NEURON](#)
 - [ACTIVATION FUNCTIONS](#)
- ☐ [NEURAL NETWORKS](#)
 - [PERCEPTRON](#)
 - [XOR FUNCTION](#)
 - [UAT AND MULTILAYER NETWORKS](#)
 - [FORWARD PROPAGATION](#)
 - [OUTPUT FUNCTION](#)
- ☐ [TRAINING](#)
 - [GRADIENT DESCENT](#)
 - [BACKPROPAGATION](#)
 - [AUTODIFF](#)
 - [BATCH AND STOCHASTIC GRADIENT DESCENT](#)
 - [CROSS-ENTROPY LOSS](#)
 - [LOSS REGULARISATION](#)

NEURONS

Artificial neural networks have been developed by **imitation of biological neurons**; it is thus worth taking a look at how they function.

Structure of a neuron



healthline

Synaptic connections or (*synapses*) act as **connecting gates** for the passage of "information" between neurons.

Dendrites are minor fibers that branch out from the cell body (*soma*) of the neuron. Through the synapses, dendrites *collect inputs from afferent neurons and propagate them towards the soma*.

The **axon** is the *main fiber* that originates from the soma and moves away from it, *bringing the output to other neurons (even distant ones)*.

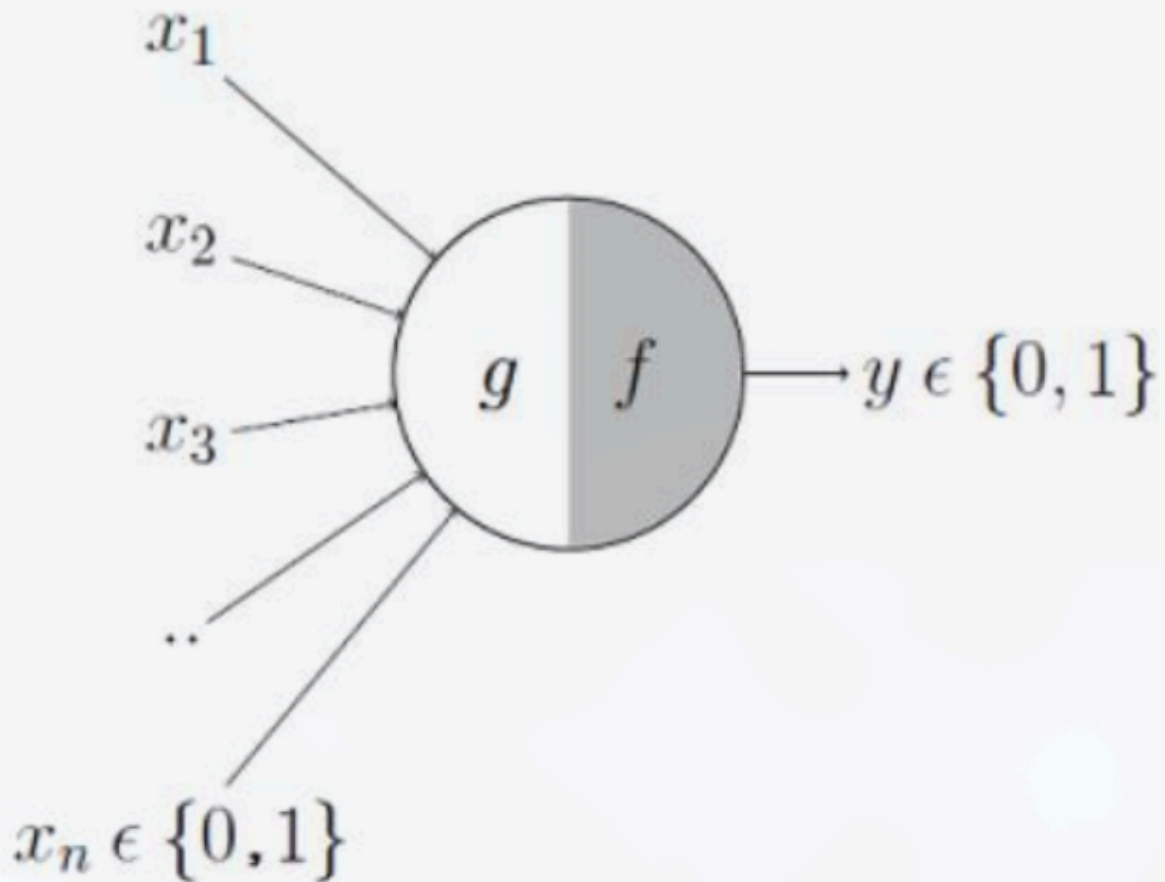
🧠 BIOLOGICAL NEURONS

A simplified explanation of a biological neuron working's is the following:

- It **receives input signals**, delivered to the dendrites.
- It **processes the inputs** in the soma.
- It **outputs a signal** and delivers it to other neurons via an axon.

MC CULLOCH - PITTS NEURON

The first **computational model** of a neuron was proposed by Warren McCulloch and Walter Pitts in 1943.



🔗 MC CULLOCH - PITTS MODEL

As we can see from the figure, the neuron may be **divided into two parts**:

1. The first, g , performs an **aggregation of the inputs**.
2. The second, f , **makes a decision** based on the aggregation.

Thus, f and g can be defined as follows:

$$g(\mathbf{x}) = g(x_1, x_2, \dots, x_n) = \sum_{i=1}^n x_i$$

$$y = f(g(\mathbf{x})) = \begin{cases} 1 & g(\mathbf{x}) \geq \theta \\ 0 & g(\mathbf{x}) < \theta \end{cases}$$

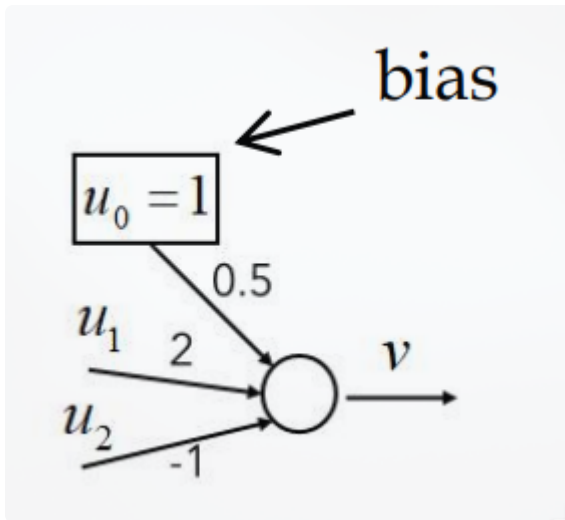
Where θ is a **threshold parameter**.

📌 NOTE

Note that, in its most general form, a neuron takes n **variable inputs** $x_1, \dots, x_n$

from **other neurons**, and a **fixed input** x_0 referred to as **bias**.

Let's now take a look at a simple example to understand what the neuron is doing:

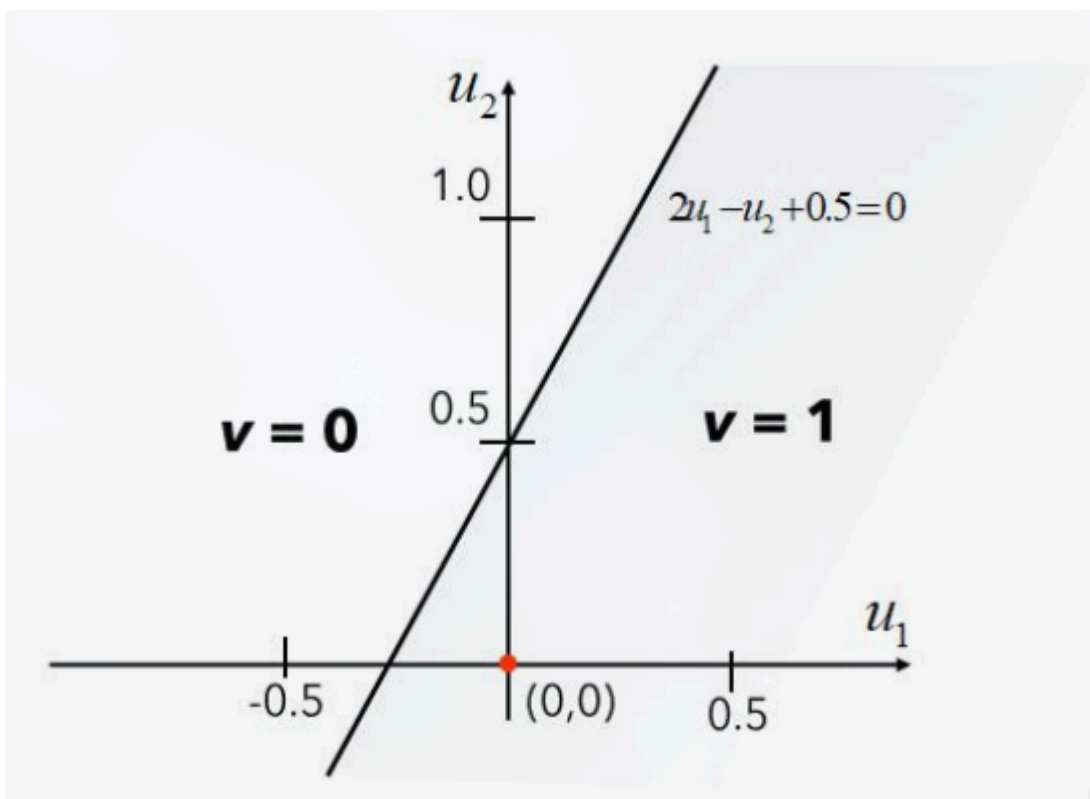


Let's assume the threshold value is $\theta = 0$ (notice that even if this was different, the reasoning shown below would still apply).

According to the definitions of f and g given above, the output v of the neuron shown in the figure will be:

$$v = \begin{cases} 1 & 2u_1 - u_2 + 0.5 \geq 0 \\ 0 & 2u_1 - u_2 + 0.5 < 0 \end{cases}$$

The neuron is **discriminating between two regions of space**, in our case $\mathbb{R}^2$ since there are *two inputs* coming from *other neurons*:



GENERALISATION

In general, if there are n input values coming from other neurons, the neuron will be **dividing** $\mathbb{R}^n$ into **two sub-regions**.

ACTIVATION FUNCTIONS

In biological neurons, when the **aggregation value** exceeds a **certain threshold**, the neuron "fires" a spike (**impulse**) and then *returns to rest*.

In the McCulloch-Pitts model, this was achieved by **comparing the weighted sum of the inputs** with the **threshold** θ .

Because this approach was found to be **too simplistic**, modern neural networks utilise a vast range of **activation functions**.

ACTIVATION FUNCTION

In an artificial neural network, an **activation function** decides the **output of a neuron** from the **weighted sum of the inputs**.

This means the neuron first computes:

$$s = (w_1 \cdot x_1) + \dots (w_n \cdot x_n) + b = \mathbf{w} \cdot \mathbf{x} + b$$

And then **outputs** a value given by the **activation function** f :

$$y = f(s)$$

NOTE

Note that:

1. A **key requirement** is that f **must be continuous and differentiable** (notice that the unit step used by McCulloch and Pitts is not differentiable): the reason will be clearer in the > [TRAINING](#) section, where we discuss the *backpropagation* algorithm.
2. Many activation functions are **non-linear**, because they can perform **more complex mappings** from the input information to the output values.

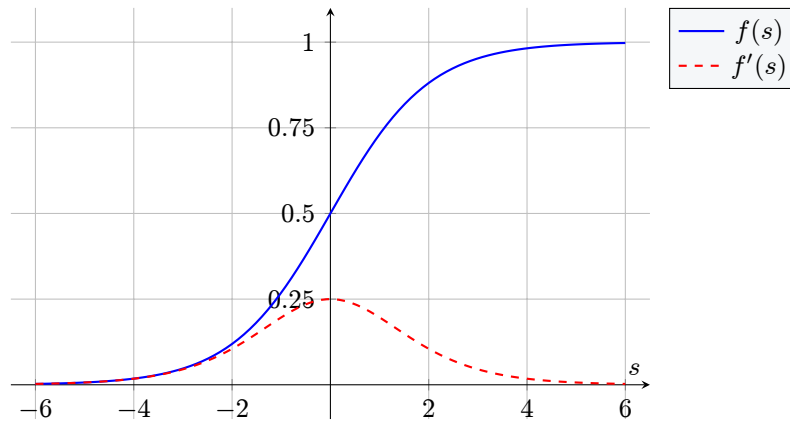
Let's take a look at two commonly used activation functions.

LOGISTIC FUNCTION (or SIGMOID)

$$f(s) = \frac{1}{1 + e^{-s}}$$

Its derivative is given by:

$$f'(s) = \frac{d}{ds} \left(\frac{1}{1 + e^{-s}} \right) = \frac{e^{-s}}{(1 + e^{-s})^2} = f(s)(1 - f(s))$$



🔍 OBSERVATION

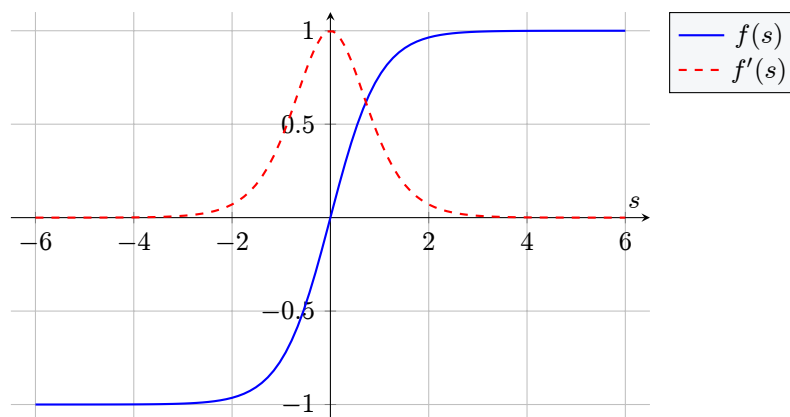
Notice how the function outputs values in the **range** $[0, 1]$, and *not just either 0 or 1* like the unit step: this is one of the characteristics that **allows more complex mappings**.

📖 HYPERBOLIC TANGENT

$$f(s) = \tanh(s) = \frac{e^s - e^{-s}}{e^s + e^{-s}}$$

And its derivative is:

$$f'(s) = \frac{d}{ds} \left(\frac{e^s - e^{-s}}{e^s + e^{-s}} \right) = 1 - \tanh^2(s)$$



NOTE

Notice that, for both of the functions presented, the **derivative approaches 0** when the **input value is particularly high**.

This introduces a problem known as **vanishing gradient** in **deep neural networks** (which shallow networks don't suffer as much).

NEURAL NETWORKS

Neural networks are made up of **groups of artificial neurons** organised in **layers**. Typically, neural networks are comprised of an **input layer**, an **output layer**, and **one or more** intermediate, or **hidden layers**: each of these levels contains one or more neurons.

Furthermore, we can distinguish between **feedforward** and **recurrent** neural networks:

FEEDFORWARD NETWORK

Neurons of a certain layer can **only be connected with neurons of a subsequent layer**.

No backwards connections or connections towards the same level are allowed.

This is by far the most widely used type of network.

RECURRENT NETWORK

This type of network **allows feedback connections** (both towards neurons of the *same layer* and of *previous layers*).

This **greatly complicates the flow of information** and training, requiring thorough consideration of the behaviour at **multiple instants of time** (*unfolding in time*).

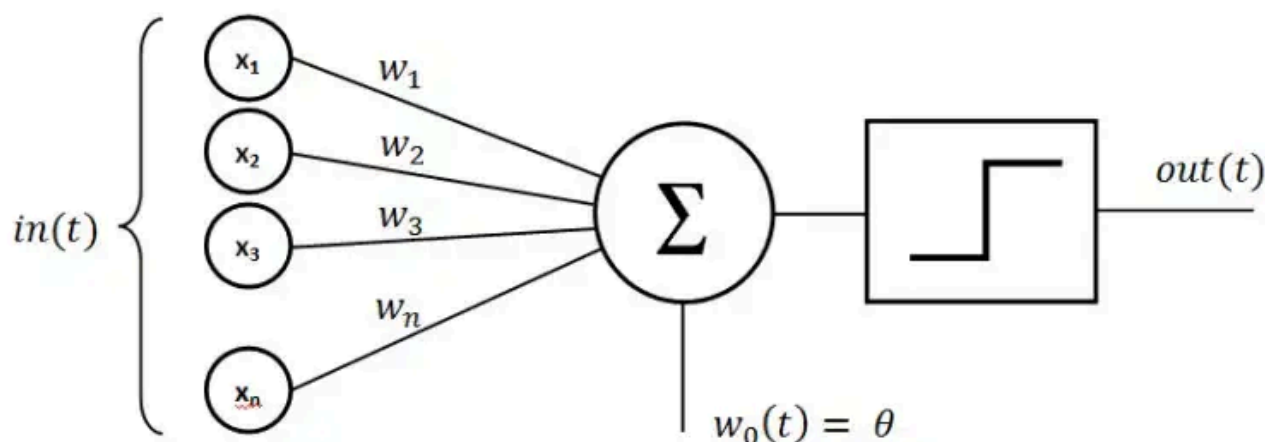
Because backward connections introduce **memory of previous states**, these networks are indicated for the **management of sequences** (e.g.audio,video, sentences in natural language) where the contribution of an element is *highly dependant on its predecessors* in the sequence.

LSTM (*Long Short-Term Memory*) are examples of such networks.

PERCEPTRON

The term **perceptron** derives from the neuron model proposed by Frank Rosenblatt in 1956. It was very similar to McCulloch and Pitts' model: it featured **weights** and a **bias** and used a **step activation function**.

The main innovation came from the fact that **weights were dynamic** and could be **updated** during a **training process**.



Rosenblatt showed that a *perceptron*, or a network with only **two levels** (we can identify an *input* and an *output layer*), can be trained using a simple rule he called **delta rule**.

📖 DELTA RULE

Each weight in the perceptron can be updated according to the following rule:

$$\Delta w_{ji} = \alpha(t_j - y_j)x_i$$

Note that the index $i = 1, \dots, n$ iterates over the *weights of the perceptron* and $j = 1, \dots, m$ iterates over the *number of training examples*.

Thus, t_j is the *expected output* for the j -th training example, y_j is the *network's output* and Δw_{ji} is the **change in the value** for weight i in relation to example j .

XOR FUNCTION

A two-level network with a linear threshold is **only able to learn linear mappings** and therefore the number of functions that can be approximated is **rather limited**.

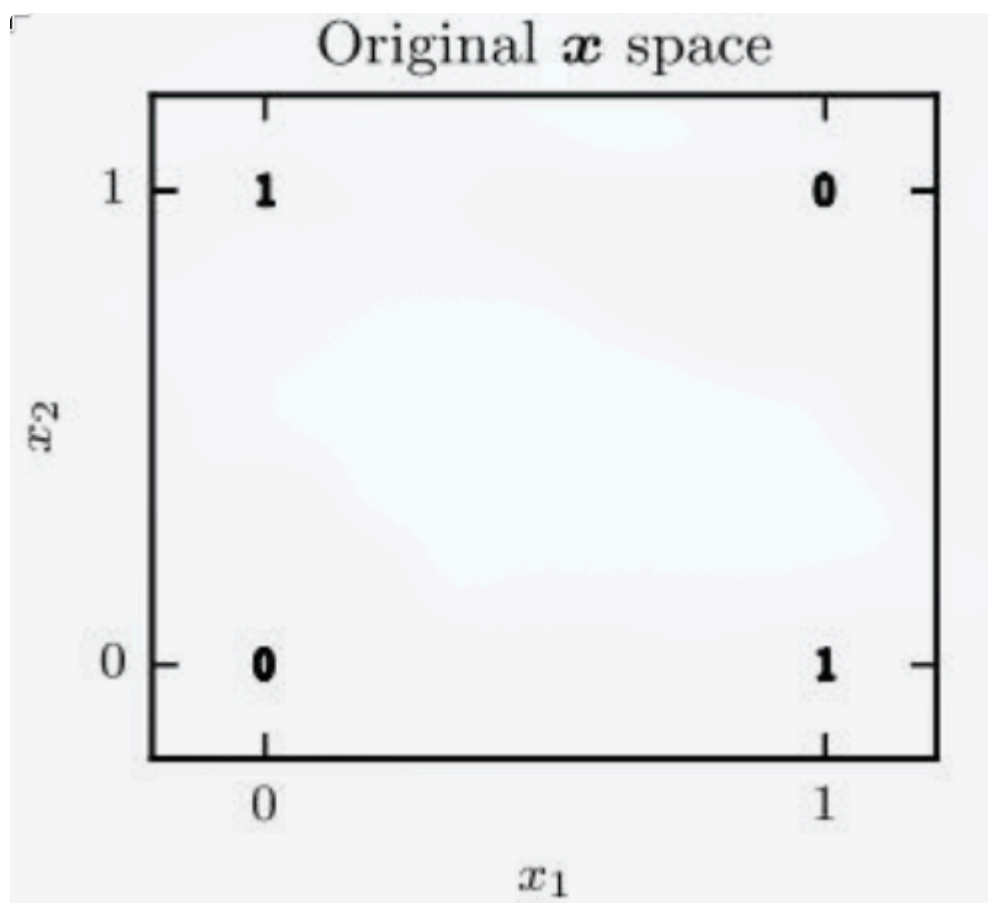
Consider, for example, the **XOR function**:

A	B	OUT
0	0	0
1	0	1
0	1	1
1	1	0

The *training set* is thus:

$$\text{Tr} = \{([0, 0], 0), ([0, 1], 1), ([1, 0], 1), ([1, 1], 0)\}$$

To understand **why** a network of this kind **can't learn this function**, we can look at the problem from the *geometric perspective* discussed in > [MC CULLOCH - PITTS NEURON](#):



🔗 GEOMETRIC INTERPRETATION

Can we **divide the plane in two regions** such that **one corresponds to the output 1** and the **other corresponds to 0**?

Looking at the figure above, it's easy to tell that, in fact, we **can not**.

UAT AND MULTILAYER NETWORKS

To learn **more complex functions** we need to use a *more complicated topology* for the network.

📖 MULTILAYER PERCEPTRON

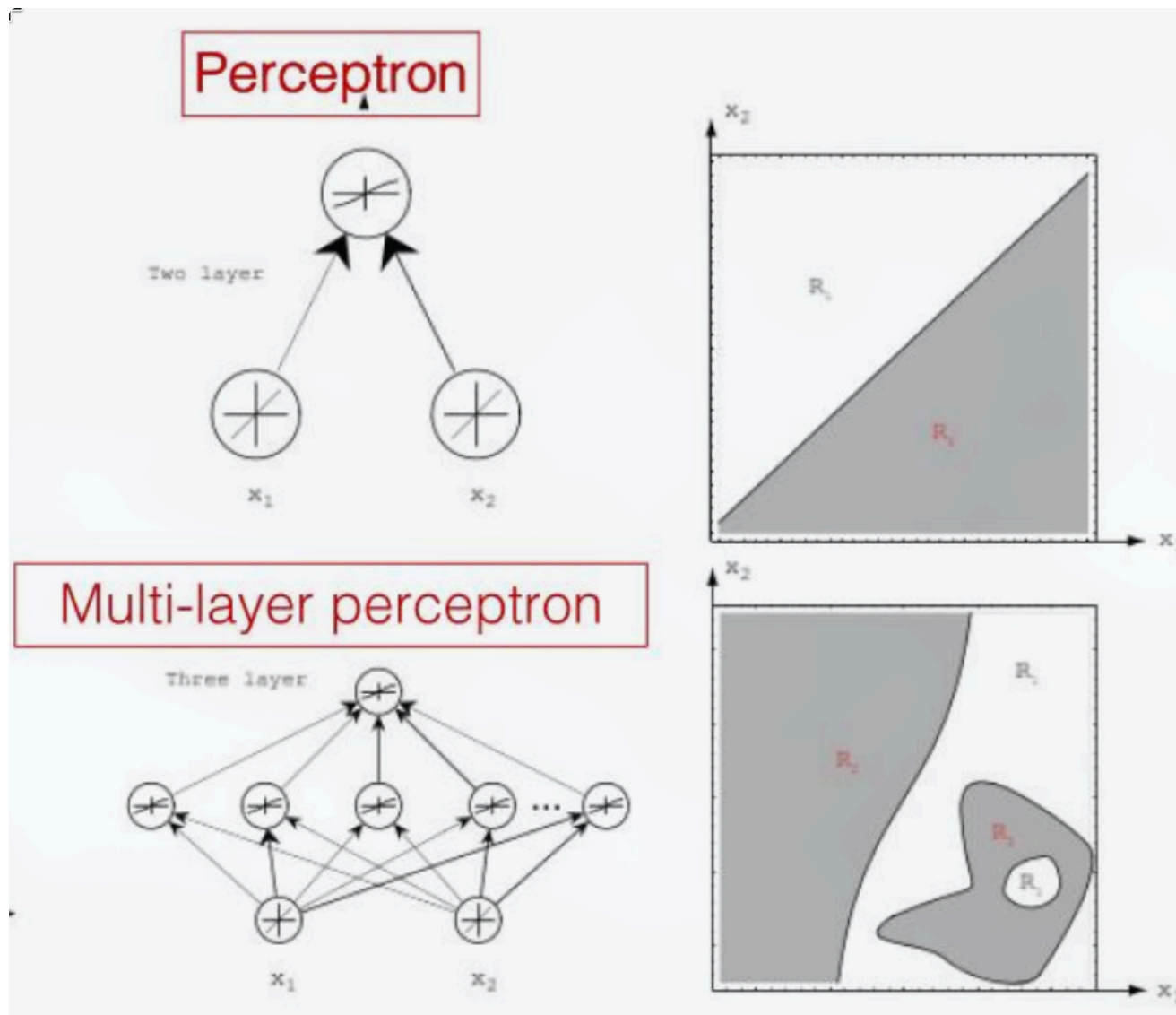
A **Multilayer Perceptron (MLP)** is a *feedforward network* with **at least 3 levels** (that is, an input and an output layer, and at least 1 hidden layer) and with **non-linear activation functions**.

We've seen that a *single neuron* with n inputs and a **linear activation function** can **discriminate between two linear subspaces** of $\mathbb{R}^n$.

By allowing the use of **non-linear** activation functions, the neuron can **discriminate two regions** separated by **more complex surfaces**.

Furthermore, if we *add more neurons and layers*, we can **combine** these **decision surfaces** to perform **complex mappings**.

The figure below illustrates this concept:



This is also the core intuition behind the **Universal Approximation Theorems (UATs)**. These theorems state that neural networks with a **certain structure** can, in principle, **approximate any continuous function to any desired degree of accuracy**.

They provide a *mathematical justification* for using neural networks, assuring that a *sufficiently large or deep network* can *model the complex, non-linear relationships* often found in real world data.

One of such theorems is of particular interest to us:

☐ UNIVERSAL APPROXIMATION THEOREM (ARBITRARY WIDTH CASE)

Let $C(X, \mathbb{R}^m)$ denote the **set of continuous functions** from a subset X of a *Euclidean* $\mathbb{R}^n$ space to a *Euclidean* space $\mathbb{R}^m$.

Let $\sigma \in C(\mathbb{R}, \mathbb{R})$.

Note that $(\sigma \circ x)_i = \sigma(x_i)$, so $\sigma \circ x$ denotes σ applied to each component of x .

Then, the following **statements are equivalent**:

1. σ is **not polynomial**.
2. For every $n, m \in \mathbb{N}$, *compact* $K \subseteq \mathbb{R}^n$, $f \in C(K, \mathbb{R}^m)$, $\varepsilon > 0$ there **exist** $k \in \mathbb{N}$, $A \in \mathbb{R}^{k \times n}$, $b \in \mathbb{R}^k$, $C \in \mathbb{R}^{m \times k}$ such that

$$\sup_{x \in K} |f(x) - g(x)| < \varepsilon$$

Where $g(x) = C \cdot (\sigma \circ (A \cdot x + b))$.

REMINDER: HEINE-BOREL THEOREM

For a **subset** S of **Euclidean space** $\mathbb{R}^n$, the following two statements are equivalent:

1. S is **compact**.
2. S is **closed and bounded**.

To better understand the theorem, the *role of every object* at play is reported below:

- σ is the **activation function** (a continuous function from $\mathbb{R}$ to $\mathbb{R}$).
- n is the **input dimension**.
- m is the **output dimension**.
- k is the **hidden layer dimension**.
- f is the **function we want to approximate**: its domain is a *compact* of $\mathbb{R}^n$.
- A is the **weight matrix** connecting the *input layer* to the *hidden layer*.
- b is the **bias vector** for the hidden layer.
- C is the **weight matrix** connecting the *hidden layer* to the *output layer*.
- $g(x)$ is **our approximation** of the target function.
- $|f(x) - g(x)|$ is the **error of our approximation**.

Now, we can give an intuitive explanation of the theorem.

UAT IN SIMPLE TERMS

First of all, notice that the theorem presents **two statements**: P (σ is non polynomial) and Q (For every $n, m \in \mathbb{N} \dots$), and tells us that they're **equivalent**. That is:

$$P \iff Q$$

This means two things:

1. If the **activation function is non-polynomial** (P true) then **Q is true**.
2. If the **activation function is polynomial** (P false), then **Q is false**.

Now we need only clarify what statement Q means.

The fundamental piece of information is that **there exists and approximation g that is arbitrarily close to our target function f .**

The final intuition lies in the definition of $g(x)$:

- It's argument is $x \in K$, therefore **its domain** is the **same** as that of **our target function f .**
- It **applies weights and biases** to x : $x \rightarrow A \cdot x + b$.
- It **runs** the computed value **through the activation function σ .**
- Finally, it computes the **output by applying the output weights.**

Thus, the theorem states that **if we have a non polynomial activation function**, then we can use it to construct a **3 layer neural network** (1 input layer with the weights contained in matrix A , 1 hidden layer with weights contained in matrix C , and 1 output layer) that **can approximate any continuous function to any desired level of accuracy.**

Furthermore, it also tells us that **if σ is polynomial**, then our network can **never be a universal approximator.**

NOTE

This theorem is one of the reasons why for a **long time** (until the explosion of deep learning) there has been a **focus on 3-level neural networks.**

On the other hand, the **theoretical existence of a solution** does **not imply** that there is an **effective way to obtain it.**

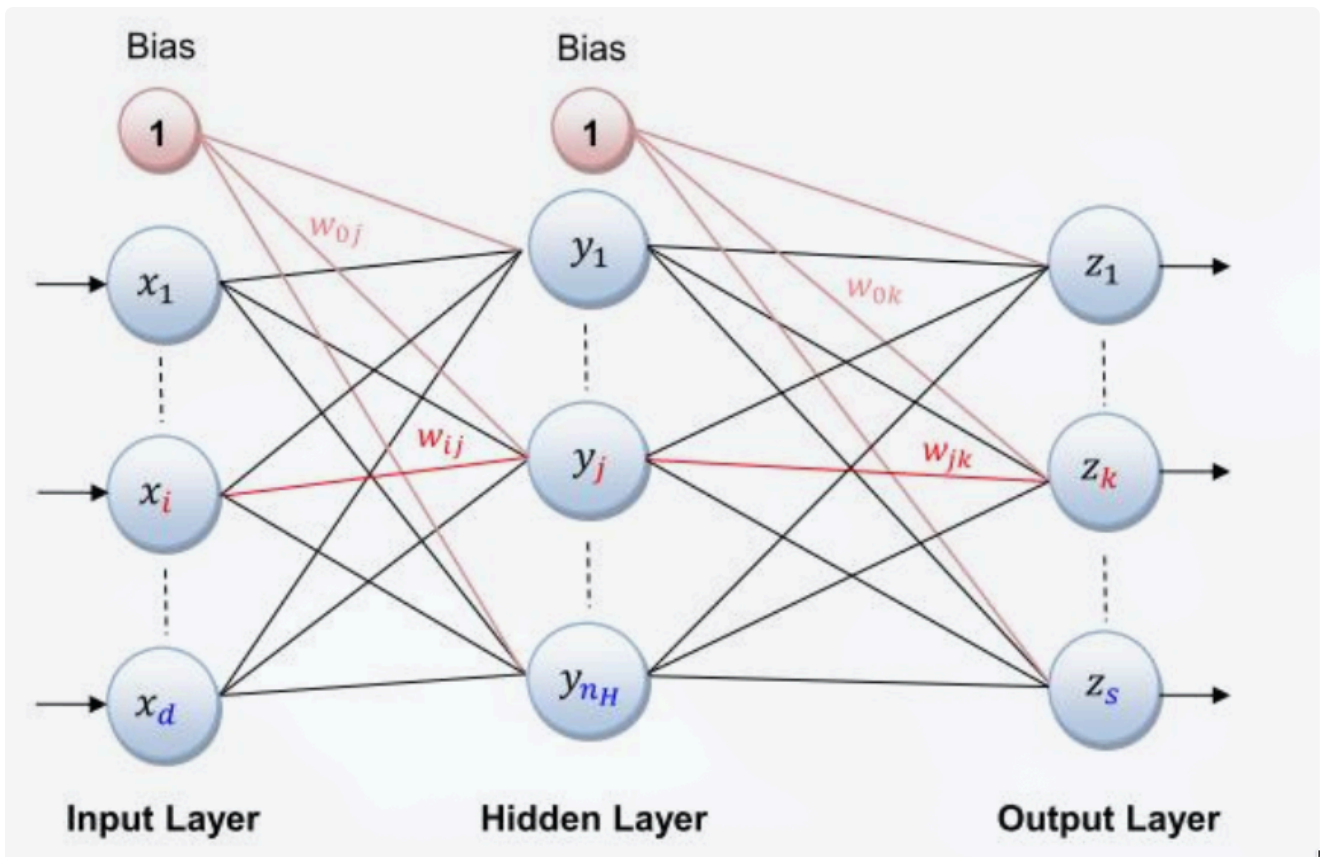
FORWARD PROPAGATION

Once the neural network has been "built" (and trained, see > [TRAINING](#)), it can be used to **generate predictions from the input data** with *forward propagation*.

FORWARD PROPAGATION

Forward propagation, or **inference**, is the *propagation of information from the input layer to the output layer.*

The reason for such name is clear when we take a look at an example:



The figure above shows a 3-level network with d input neurons, n_H hidden neurons (organised in a single layer) and s output neurons. Notice that the total number of parameters is given by $d \cdot n_H + n_H \cdot s + n_H + s$.

If we denote the activation function with f , the k -th **output value** is given by:

$$z_k = f \left(\sum_{j=1}^{n_H} w_{jk} \cdot y_j + w_{0k} \right)$$

But each y_j is also computed in a similar fashion.

Thus, we obtain:

$$z_k = f \left(\sum_{j=1}^{n_H} w_{jk} \cdot f \left(\sum_{i=1}^d w_{ij} \cdot x_i + w_{0j} \right) + w_{0k} \right)$$

🔍 OBSERVATION

Therefore, to **compute the output value** of a neuron, we **first need to know all the output values** of the **neurons in the previous layer**.

✍️ CLARIFICATION

Notice that in neural networks, neurons in the **input layer** usually **do not apply an activation function**.

They simply **receive the input** (pre-processed) raw data and **pass it** to the first hidden layer, without performing calculations.

OUTPUT FUNCTION

The **output of the network** depends on the **activation function** utilised by the **output layer**.

Notice that if we use $\tanh$, the outputs are in the range $[-1, 1]$ and thus they **are not probabilities**.

On the other hand, **sigmoid** output values are in the **correct range** $[0, 1]$, but there is **no guarantee** that they **sum to one**, which is a fundamental requirement if we want to interpret them as probabilities.

When a neural network is used as a **multi-class classifier**, the use of the **softmax activation function** allows us to transform the net values produced by the last level of the network into **class probabilities**.

SOFTMAX

The output of the k -th output neuron is given by:

$$z_k = \text{softmax}(\text{net}_k) = \frac{e^{\text{net}_k}}{\sum_{c=1}^s e^{\text{net}_c}}$$

Where net_c is the *input value* for the i -th neuron.

These values can **indeed be interpreted as probabilities** since $z_k \in [0, 1]$, and they **sum to one**.

TRAINING

Although the *first artificial neurons date back to the 1940s*, effective **training algorithms** were **not available until the mid-1980s**.

In 1986 *Rumelhart, Hinton & Williams* introduced the **Error Backpropagation** algorithm, arousing great attention in the scientific community.

At its core, it is actually the *application of the chain derivation rule*, an idea that nobody had successfully applied in this context for decades.

Before diving into the details, we first need to understand what **training a network** means.

Once the **topology** (number of levels and neurons) has been **fixed**, the **training** of a neural network consists in **finding the values of the weights** that determine the **desired mapping** between input and output.

The desired mapping depends on the problem we're interested in solving:

- If we're training a **classifier**, the desired output is the **correct class label** of the input pattern.
- If we're solving a **regression** problem, the desired output is the **correct prediction of the dependent variable** corresponding to the input value of the independent variable.

To find the **optimal values of the weights** we need to **minimise a loss function**, which tells us **how far our predictions are from the expected results**.

We thus define the *loss function* $J(\mathbf{w}, \mathbf{x})$:

LOSS FUNCTION

The loss function (error) **for pattern $\mathbf{x}$** is defined as follows:

$$J(\mathbf{w}, \mathbf{x}) := \sum_{i=1}^s (t_i - z_i)^2$$

Where s is the *number of output neurons*, t_i is the *desired output value* for the i -th output neuron and z_i is the *actual output* of the neuron.

Notice that this is a function of the **weights $\mathbf{w}$** of the neural network, and the **individual pattern $\mathbf{x}$** .

The **training error** is the average of $J(\mathbf{w}, \mathbf{x})$ on **all the patterns in the training set**:

TRAINING ERROR

$$J(\mathbf{w}) := \frac{1}{N} \sum_{j=1}^N J(\mathbf{w}, \mathbf{x}_j)$$

Where N is the *number of training patterns*.

NOTE

Note that there are **different possible loss functions**.

The one presented above is amongst the most widely used, though **cross-entropy loss** (see > [CROSS-ENTROPY LOSS](#)) is deemed *more accurate*.

GRADIENT DESCENT

We've seen that **training** a network boils down to **optimising a function**, which is the **training error $J(\mathbf{w})$** .

Notice that $J(\mathbf{w}) = J(w_1, \dots, w_T)$ where T is the *total number of weights in the network*. Since we are optimising a function $f : \mathbb{R}^T \rightarrow \mathbb{R}$, we first need to set its **gradient equal to**

zero and solve T equations to find stationary points:

$$\nabla J(\mathbf{w}) = 0$$
$$\left\{ \begin{array}{l} \frac{\partial J}{\partial w_1} = 0 \\ \dots \\ \frac{\partial J}{\partial w_T} = 0 \end{array} \right.$$

We then need to find **which of these are minima** (e.g. by using the Hessian matrix criterion).

Notice that for **simple models** (like *linear regression*) this system of equations **can be solved analytically**.

💡 LIMITATIONS OF THE ANALYTICAL APPROACH

When the neural network has a **large amount of weights** (several hundreds of billions to trillions), it's clear that we **can't solve for minima analytically**.

We thus need a **numerical method** to approach the problem.

One such method is **gradient descent**:

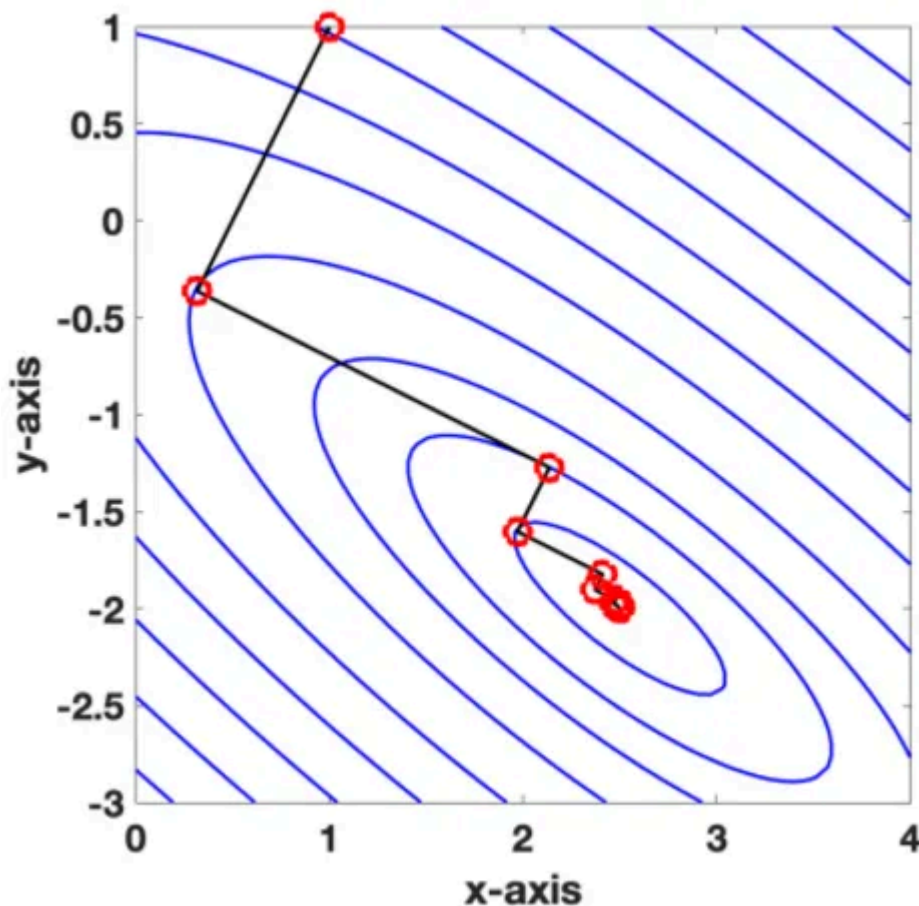
📖 GRADIENT DESCENT

Gradient descent is an **iterative numerical method** to find **(local) minima** of functions.

In the case of *neural network training*, at **each step** we compute:

$$\mathbf{w}_{\text{new}} = \mathbf{w}_{\text{curr}} - \eta \cdot \nabla J(\mathbf{w}_{\text{curr}})$$

Where η is the **learning rate**.



🔍 EXPLANATION

Remember that $\nabla J(\mathbf{w}_{\text{curr}})$ tells us the **direction of maximum growth** of the function. Since we are interested in **minimising** J , we move in the **opposite direction** (which we know is the *direction of maximum decline*).

Thus, at **each iteration**, we make a **small step** that takes us closer to the weight values that minimise the training loss.

There are at least two questions left to answer at this point:

1. *When* do we stop?
2. *How often* do we perform these steps?

The answer to the first question is relatively simple: we **define a stopping criterion** and stop the algorithm when it is satisfied.

A stopping criterion may be as simple as **repeating the steps a fixed number of times**, continuing **until a certain performance threshold** is reached on a *validation set*, or continuing until the **gradient is small enough** to assume we've reached a minimum.

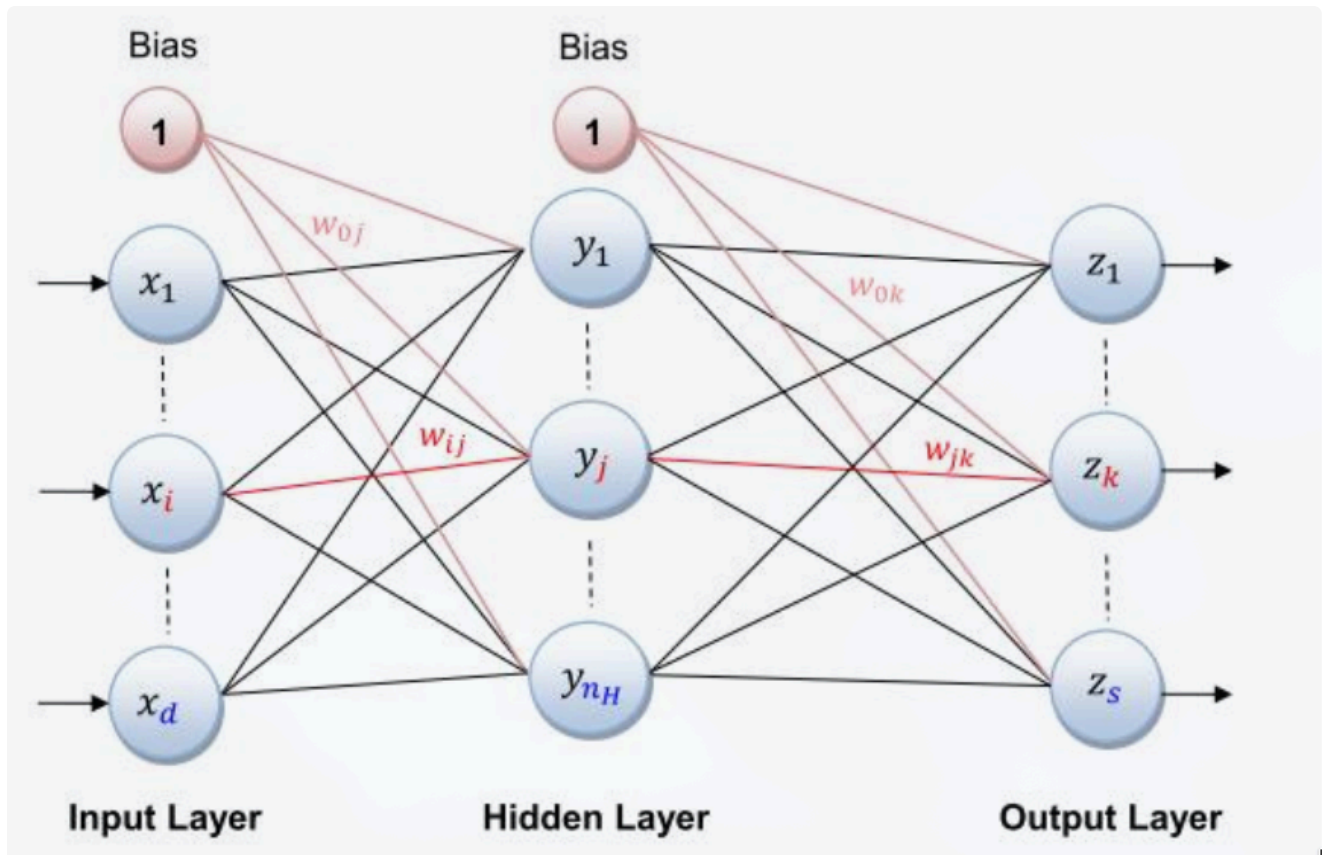
The answer to the second question depends on the **specific implementation** of gradient descent: earlier versions performed *one weight update after going through the entire training set*; modern approaches **divide the training set in batches**, update the weights after each batch has been processed, and then repeat the process multiple times (see > [BATCH AND STOCHASTIC GRADIENT DESCENT](#) for details).

BACKPROPAGATION

Another question that should come out naturally when looking at the **gradient descent** approach is **how do we compute the gradient of J** ?

To do so, we must **propagate the error backwards** in the network.

To understand how and why, let's take a look at an example network with 3 layers:



Remember that **training loss** was the **average loss** across all patterns of the training set:

$$J(\mathbf{w}) = \frac{1}{N} (J(\mathbf{w}, x_1) + \dots + J(\mathbf{w}, x_N))$$

We thus have:

$$\nabla J(\mathbf{w}) = \frac{1}{N} (\nabla J(\mathbf{w}, x_1) + \dots + \nabla J(\mathbf{w}, x_N))$$

Since the gradient of the overall loss function is given by the sum of the gradients for the **loss of each training pattern**, we can focus our attention on just one of these, which we will call J for the sake of simplicity:

$$J = J(\mathbf{w}, x) = \frac{1}{2} \sum_{c=1}^s (t_c - z_c)^2$$

Notice that the factor of $1/2$ has been *added for convenience*.

We begin by looking at the partial derivatives **with respect to the weights connecting the hidden and the output layer**.

For each $k = 1, \dots, s$ we have:

$$\begin{aligned}
\frac{\partial J}{\partial w_{jk}} &= \frac{\partial}{\partial w_{jk}} \left(\frac{1}{2} \sum_{c=1}^s (t_c - z_c)^2 \right) \\
&= \frac{\partial}{\partial w_{jk}} \frac{1}{2} ((t_1 - z_1)^2 + \dots + (t_k - z_k)^2 + \dots + (t_s - z_s)^2) \\
&= (t_k - z_k) \frac{\partial}{\partial w_{jk}} (-z_k)
\end{aligned}$$

Notice that only z_k depends on w_{jk} , and, in particular, we have:

$$z_k = f(\text{net}_k)$$

Where f is the **activation function** of our network and net_k is given by:

$$\text{net}_k = \sum_{a=1}^{n_H} (w_{ak} y_a) + w_{0k}$$

Because $w_{jk} y_j$ is the *only term that depends on w_{jk}* , we have:

$$\begin{aligned}
\frac{\partial J}{\partial w_{jk}} &= -(t_k - z_k) \frac{\partial f(\text{net}_k)}{\partial w_{jk}} \\
&= -(t_k - z_k) \frac{\partial f(\text{net}_k)}{\partial \text{net}_k} \frac{\partial \text{net}_k}{\partial w_{jk}} \\
&= -(t_k - z_k) f'(\text{net}_k) y_j
\end{aligned}$$

Notice how we **applied the chain rule**, which is the **core idea** behind **backpropagation**.

We can define $\delta_k := (t_k - z_k) f'(\text{net}_k)$ and write:

$$\frac{\partial J}{\partial w_{jk}} = -\delta_k y_j$$

🔍 UPDATING THE FINAL LAYER OF WEIGHTS

Therefore each weight w_{jk} in the final layer can be **updated as follows**:

$$w_{jk} = w_{jk} + \eta(\delta_k y_j)$$

With $j = 1, \dots, n_H$.

Let's now look at the partial derivatives with respect to the **weights between the input and hidden layers**.

$$\begin{aligned}
\frac{\partial J}{\partial w_{ij}} &= \frac{\partial}{\partial w_{ij}} \left(\frac{1}{2} \sum_{c=1}^s (t_c - z_c)^2 \right) \\
&= - \sum_{c=1}^s (t_c - z_c) \frac{\partial z_c}{\partial w_{ij}} \\
&= - \sum_{c=1}^s (t_c - z_c) \frac{\partial z_c}{\partial \text{net}_c} \frac{\partial \text{net}_c}{\partial w_{ij}} \\
&= - \sum_{c=1}^s (t_c - z_c) f'(\text{net}_c) \frac{\partial \text{net}_c}{\partial w_{ij}}
\end{aligned}$$

Notice that now **all the terms** of the summation **depend on** the weight w_{ij} because the output of the j -th hidden neuron is **fed to every output neuron**.

We have:

$$\begin{aligned}
\frac{\partial \text{net}_c}{\partial w_{ij}} &= \frac{\partial}{\partial w_{ij}} \left(w_{0c} + \sum_{r=1}^{n_H} w_{rc} y_r \right) \\
&= \frac{\partial}{\partial w_{ij}} \sum_{r=1}^{n_H} w_{rc} f(\text{net}_r)
\end{aligned}$$

Since only the term for $r = j$ depends on w_{ij} we obtain:

$$\begin{aligned}
&= \frac{\partial}{\partial w_{ij}} (w_{jc} f(\text{net}_j)) \\
&= w_{jc} \frac{\partial f(\text{net}_j)}{\partial \text{net}_j} \frac{\partial \text{net}_j}{\partial w_{ij}} \\
&= w_{jc} f'(\text{net}_j) \frac{\partial}{\partial w_{ij}} \sum_{q=1}^d w_{jq} x_q \\
&= w_{jc} f'(\text{net}_j) x_i
\end{aligned}$$

And we thus have:

$$\begin{aligned}
\frac{\partial J}{\partial w_{ij}} &= - \sum_{c=1}^s \underbrace{(t_c - z_c) f'(\text{net}_c)}_{:= \delta_c} w_{jc} f'(\text{net}_j) x_i \\
&= - \underbrace{\left(f'(\text{net}_j) \sum_{c=1}^s \delta_c w_{jc} \right)}_{:= \delta_j} x_i
\end{aligned}$$

So we conclude:

$$\frac{\partial J}{\partial w_{ij}} = -\delta_j x_i$$

🔗 UPDATING THE FIRST LAYER OF WEIGHTS

Therefore, each weight w_{ij} in the first layer can be **updated as follows**:

$$w_{ij} = w_{ij} + \eta(\delta_j x_i)$$

✍ CLARIFICATIONS

1. The **weights** related to the **biases** are updated using the **same rules**, with their **input sources** (y_0 and x_0 in our example) **set to one**.
2. In the example, we used **MSE** as our choice of **loss function**. We have already stated that **other loss functions** also exist, but the **only difference** in the algorithm would be in the $(t_c - z_c)$ terms, which originate from the *first differentiation of J* .
In general, we can denote these terms with $\partial_{z_c} J$; thus, with $\nabla_z J$ we mean the vector $(\partial_{z_1} J, \dots, \partial_{z_s} J)$.

The core concept to understand from the example above is that, **to compute the gradient** with respect to the **weights of a given level l** , we **first need to compute it** with respect to the **weights of the successive level $l + 1$** .

This is due to the fact that **each update step requires** a $\delta_i^{(l)}$ parameter that is computed as follows:

✍ BACKPROPAGATION IN GENERAL

For the **output layer L** :

$$\delta_i^{(L)} = \partial_{z_i} J f'(\text{net}_i^{(L)}), i = 1, \dots, s$$

For the **hidden layers $1 < l < L$** :

$$\delta_i^{(l)} = \left(\sum_{j=1}^{N_{l+1}} w_{ij}^{(l)} \delta_j^{(l+1)} \right) f'(\text{net}_i^{(l)}), i = 1, \dots, N_l$$

Since every layer has **one of these parameters for each neuron**, we can represent them with **vectors** (one per layer) and write the above equations in the following, more **compact forms**:

$$\delta^{(L)} = \nabla_z J \odot f'(\text{net}^{(L)})$$

$$\delta^{(l)} = \left(w^{(l)} \delta^{(l+1)} \right) \odot f'(\text{net}^{(l)})$$

Where $w^{(l)} \in \mathbb{R}^{N_l \times N_{l+1}}$ is the **weight matrix** with w_{ij}^l connecting neuron i in layer l with neuron j in layer $l + 1$, and $\odot$ denotes *element-wise* multiplication.

NOTE: some textbooks use w_{ji} to indicate the weight going from neuron i in layer l to neuron j in layer $l + 1$; the index convention is the opposite of ours.

AUTODIFF

Now that we know the theory behind *backpropagation*, we need to understand **how neural networks (computers) can calculate the partial derivatives of a function**.

The answer lies in a process known as **automatic differentiation**, or **autodiff**.

🔗 AUTO DIFF

The intent of automatic differentiation is to **obtain the numerical value of a derivative by completely avoiding any symbolic computation**.

This is achieved by leveraging **intermediate variables** present in the **original function implementation**: the idea is that, ultimately, the differentiable functions that come up in deep learning are composed of **underlying primitive operations** whose **derivatives are known**, and the **chain rule allows us to compose these together**.

For a start, consider the following simple example:

$$f(x, y, z) = (x + y)z$$

Notice that this expression can be broken down into the following **primitive operations**:

$$q = x + y$$

$$f = qz$$

Moreover, we **know how to compute the derivatives** of both expressions separately:

$$\frac{\partial q}{\partial x} = 1$$

$$\frac{\partial q}{\partial y} = 1$$

$$\frac{\partial f}{\partial q} = z$$

$$\frac{\partial f}{\partial z} = q$$

Furthermore, the **chain rule** tells us that the correct way to *chain* these gradient expressions together is through multiplication. For example:

$$\frac{\partial f}{\partial x} = \frac{\partial f}{\partial q} \frac{\partial q}{\partial x} = z \cdot 1 = z$$

In practice this is simply a multiplication of the two numbers that hold the two gradients. Lets see this with an example:

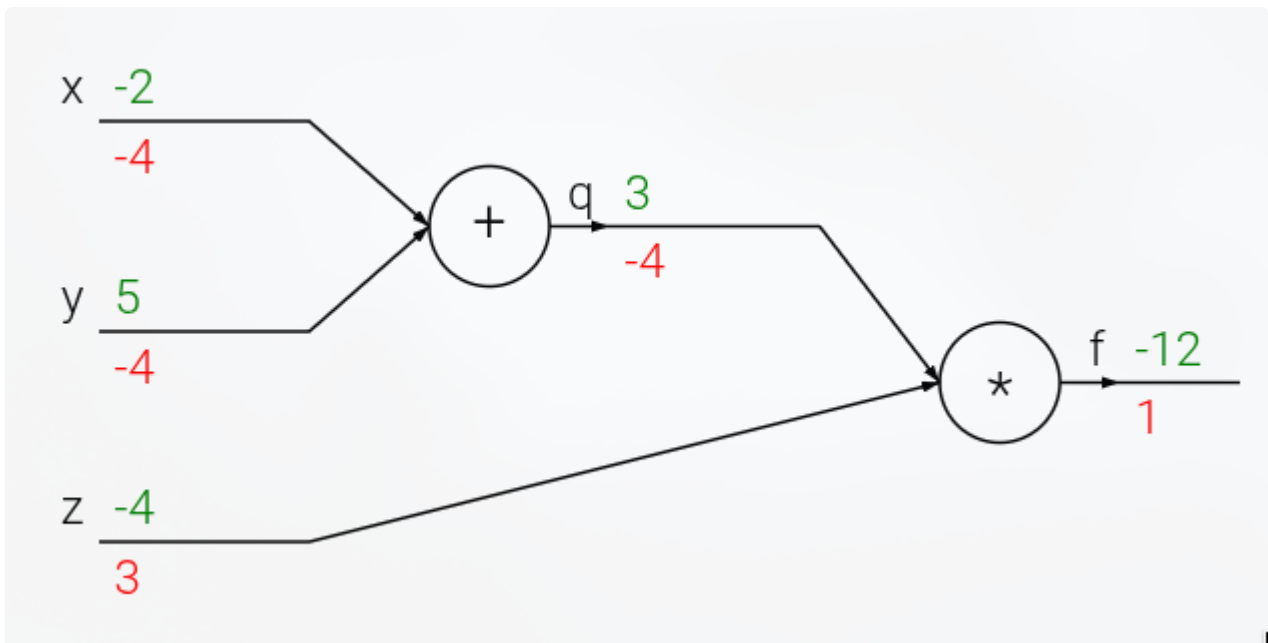
```
# Example inputs
x = -2; y = 5; z = -4

# Forward pass
q = x + y
f = q * z

# Backpropagation (reverse order compared to the forward pass)
# First we backprop through f = q * z
dfdq = q
dfdq = z
# Now backprop through q = x + y
dqdx = 1.0
dqdy = 1.0
# Finally we can compute the gradient
dfdx = dfdq * dqdx
dfdy = dfdq * dqdy
```

At the end, the gradient is given by `[dfdx, dfdy, dfdz]` .

This computation can also be nicely visualized with a circuit diagram:



NOTE

Going forward, we will use a more concise notation that omits the df prefix. For example, we will simply write dq instead of $dfdq$.

In practice, any kind of differentiable function can act as a gate, and we can group multiple gates into a single gate, or **decompose a function into multiple gates** whenever it is convenient.

Let us look at another expression that illustrates this point:

$$f(w, x) = \frac{1}{1 + e^{-(w_0x_0 + w_1x_1 + w_2x_2)}}$$

Notice that the function can be **broken down into the following blocks**:

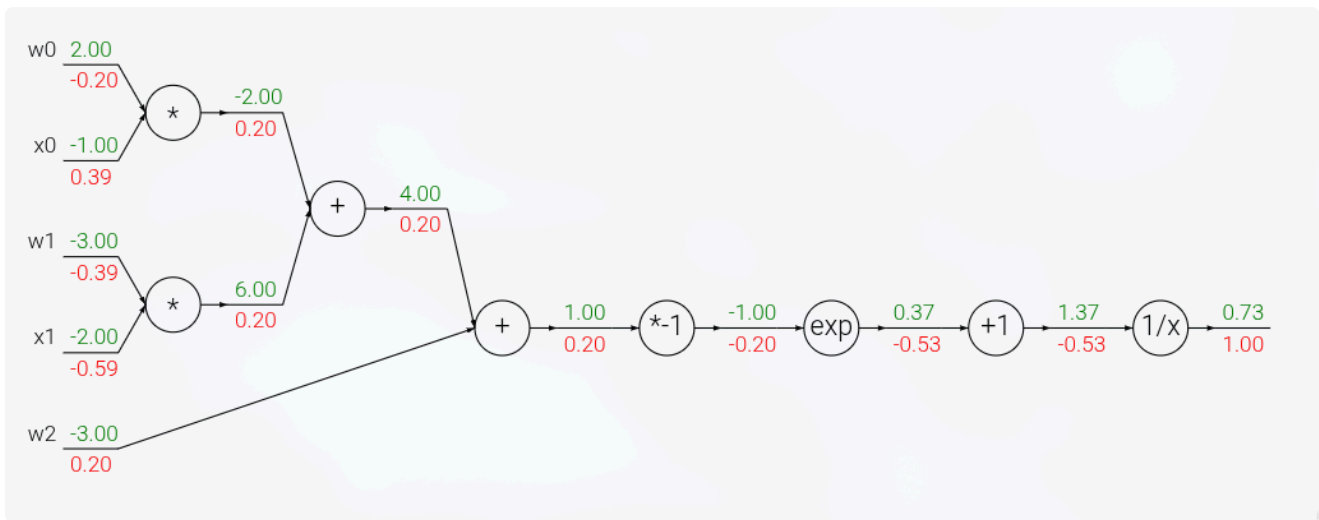
$$f(x) = \frac{1}{x} \quad \rightarrow \quad \frac{df}{dx} = -\frac{1}{x^2}$$

$$f_c(x) = c + x \quad \rightarrow \quad \frac{df}{dx} = 1$$

$$f(x) = e^x \quad \rightarrow \quad \frac{df}{dx} = e^x$$

$$f_a(x) = ax \quad \rightarrow \quad \frac{df}{dx} = a$$

Assuming input values $w = [2.0, -3.0, -3.0]$ and $x = [-1.0, -2.0]$, the circuit looks as follows:



Notice that the above example is an application of the **sigmoid** activation function. We already know that its derivative is simply given by:

$$\frac{d\sigma(x)}{dx} = (1 - \sigma(x))\sigma(x)$$

Therefore, in any **real practical application** it would be very useful to **group the above operations** into a **single gate**.

Let us take a look at the code for this example:

```
# Example inputs
w = [2,-3,-3]
x = [-1,-2]

# Forward pass
dot = w[0] * x[0] + w[1] * x[1] + w[2]
f = 1.0 / (1 + math.exp(-dot))

# Backward pass
ddot = (1 - f) * f
dx = [w[0] * ddot, w[1] * ddot]
dw = [x[0] * ddot, x[1] * ddot, 1.0 * ddot]    # result we care about
```

💡 INSIGHTS ON IMPLEMENTATION

In practice it is always helpful to break down the forward pass into stages that are easily backproped through.

Notice that **which parts** of the forward function **we think of as gates** is a matter of convenience.

It helps to be aware of which parts of the expression have **easy local gradients**, so that they can be **chained together** with the **least amount of code** and effort.

To understand these tips, consider the following example:

$$f(x, y) = \frac{x + \sigma(y)}{\sigma(x) + (x + y)^2}$$

Notice that this function is *practically useless*, but it is a *good example*.

The resulting code is:

```
# Example inputs
x = 3
y = -4

# Forward pass
sigy = 1.0 / (1 + math.exp(-y))           #(1)
num = x + sigy                             #(2)
sigx = 1.0 / (1 + math.exp(-x))           #(3)
xpy = x + y                               #(4)
xpysqr = xpy**2                           #(5)
den = sigx + xpysqr                       #(6)
invden = 1.0 / den                        #(7)
f = num * invden                          #(8)
```

Notice that we have structured the code in such way that it contains multiple intermediate variables, each of which are only simple expressions for which we already know the local gradients.

```
# backprop f = num * invden
dnum = invden                             #(8)
dinven = num                             #(8)
# backprop invden = 1.0 / den
dden = (-1.0 / (den**2)) * dinven        #(7)
# backprop den = sigx + xpysqr
dsigx = (1) * dden                       #(6)
d xpysqr = (1) * dden                    #(6)
# backprop xpysqr = xpy**2
d xpy = (2 * xpy) * d xpysqr            #(5)
# backprop xpy = x + y
dx = (1) * d xpy                         #(4)
dy = (1) * d xpy                         #(4)
# backprop sigx
dx += ((1 - sigx) * sigx) * dsigx        #(3)
# backprop num = x + sigy
dx += (1) * dnum                         #(2)
dsigy = (1) * dnum                       #(2)
```

```
# backprop sigy
dy += ((1 - sigy) * sigy) * dsigy # (1)
```

🔗 CACHING VARIABLES

To compute the backward pass it is very helpful to have some of the **variables that were used in the forward pass**, that is, we want to structure our code so that we **cache these variables** to have them available during backpropagation. In the example above, we reused `num`, `invden`, `sigx` and `sigy` from the forward pass.

✎ GRADIENTS ADD UP

When we perform backpropagation we must be careful to use `+=` instead of `=` to **accumulate the gradient** to account for *all possible branches* in the circuit.

The above sections were concerned with single variables, but all concepts extend in a straight-forward manner to **matrix and vector operations**.

However, one must pay closer attention to dimensions and transpose operations.

```
# Random weights and inputs
W = np.random.randn(5, 10)
X = np.random.randn(10, 3)
# Forward pass
D = W.dot(X)

# Suppose we have the gradient on D from above
dD = np.random.randn(*D.shape) # Same shape as D

# Compute the gradients (.T gives the transpose of the matrix)
dW = dD.dot(X.T) # Same shape as W
dX = W.T.dot(dD) # Same shape as X
```

BATCH AND STOCHASTIC GRADIENT DESCENT

We showed how **gradient descent** and **backpropagation** work in the above paragraphs, so the only detail left to discuss is **how to use the training set**.

Many implementations of gradient descent exist and they differ by **how they split the training set**:

📋 GD IMPLEMENTATIONS

1. **BATCH GD**: It uses the **entire training set** to compute the gradient and **updates the weights once every time it processes the entire set**.
2. **STOCHASTIC GRADIENT DESCENT (SGD)**: only **one training pattern** is **sampled at random** to compute the gradient and the **weights are updated after each pattern is processed**.
3. **MINI BATCH SGD**: a version of *SGD* that uses **small subsets** of the training set (e.g. 32-256 samples) for each gradient computation *instead of just one pattern*.

Note that in the context of neural network training, we call **epoch** a "**full pass**" of the training set, and **iteration** a single **update of the weights**.

Thus, if the training set is composed of N patterns:

1. Batch GD performs **1 iteration per epoch**.
2. SGD performs **N iterations per epoch**.
3. Mini-batch SGD performs $N/\text{batch size}$ **iterations per epoch**.

Using **Batch GD** guarantees **smooth steps** and **convergence** because the *gradient is accurately computed over the whole training set*; on the other hand, this variant is **prone to getting trapped in local minima and saddle points**.

Thus, the idea behind **Stochastic GD** was to **introduce noise**, and thus make the gradient "bounce" around erratically, to help **escape local minima**.

Using **mini-batches** allows to **balance stability and noise**: the algorithm is stable enough to **converge smoothly**, but introduces **enough noise to avoid local minima**.

It's also worth noting that **Mini Batch SGD** represents the perfect sweetspot in terms of **computational efficiency**. This is because **Batch GD** would require the **entire dataset** to be **loaded in memory**, which is *impossible for large training sets*, while **SGD cannot utilise modern GPU parallelisation** effectively, since matrix multiplications for single samples are very inefficient.

CROSS-ENTROPY LOSS

Cross-entropy loss is the modern standard in the context of **loss functions**, especially for **multi-class classification** problems.

Cross-entropy loss for deep neural network training can be defined as follows:

📖 CROSS-ENTROPY LOSS

Let p and q be two **discrete probability distributions** with support $\mathcal{X}$: p being the **target** and q the **network output**.

Then the *cross-entropy loss* of the network $H(p, q)$ is given by:

$$H(p, q) = - \sum_{x \in \mathcal{X}} p(x) \log q(x)$$

NOTE: some textbooks may call this **log loss** or **multinomial logistic loss**.

🔗 INTERPRETATION

Cross-entropy is a concept that originates in **information theory** and, to understand what it truly represents, it's better to first introduce *two more core concepts* of this field.

INFORMATION: Given a "message" m (think of a long streak of *symbols*) where each *symbol* $s_i \in m$ appears with *probability* $p(s_i)$, the **information carried by the symbol** is given by:

$$I(s_i) = -\log_2 p(s_i)$$

This has an interesting interpretation: it is the **minimum number of bits** needed to **encode the symbol**.

ENTROPY: With the definition of information given above, we can consider the **average number of bits** needed to **encode each symbol** in the message, which is simply given by a weighted average of the information over all the symbols:

$$H(m) = - \sum_{s_i \in m} p(s_i) \log_2 p(s_i)$$

Thus, we can think of entropy as the **average amount of information carried by each symbol in a message**.

Finally, we can give an interpretation of **cross entropy**.

Notice that $H(p, q)$ is still a **weighted average**, just like **entropy**; the only difference is that we are *encoding symbols* using the **number of bits** calculated **according to our network, instead of the target distribution**.

Thus, cross-entropy essentially answers the question:

*If we were to **represent a message** using a **sub-optimal encoding**, how many **bits** would we **need on average**?*

It's now intuitive to understand that the **difference** between $H(p, q)$ (sub-optimal value) and $H(p)$ (optimal value) tells us **how many bits we are wasting on average** to encode the message and is known as **Kullback-Leibler divergence**:

$$D_{\text{KL}}(p||q) = H(p, q) - H(p)$$

Notice that $D_{\text{KL}}(p||q)$ is **minimum (0) when** $H(p, q) = H(p)$; thus, minimising $H(p, q)$ is **equivalent** to minimising $D_{\text{KL}}(p||q)$

Lastly, a clarification on the *log* used: the definition from information theory (reflecting the intuitions explained above) use *log base 2*, while in machine learning we use the **natural log**. The two only differ by a constant factor and the intuitions are still useful knowledge.

Notice that, when the **target vector** takes a **one-hot form** (every component is 0, except a single 1 at the i -th position, such as $t = [0, 0, \dots, 1, \dots, 0]$), the **cross-entropy loss** for our prediction vector z is simply given by:

$$H(t, z) = - \sum_j t_j \log z_j = - \log z_j$$

If z_j is obtained via the **softmax** activation function, then:

$$H(t, z) = - \log \left(\frac{e^{\text{net}_j}}{\sum_{c=1}^s e^{\text{net}_c}} \right) = -\text{net}_j + \log \sum_{c=1}^s e^{\text{net}_c}$$

LOSS REGULARISATION

To **reduce the risk of overfitting** the training set of a neural network with a large number of parameters (weights), we can use **regularisation techniques**.

In particular, regularisation is **especially important** when the **training set is not large in comparison to the complexity of our model**.

It's been shown that neural networks whose **weights**, or a **subset** of them, take **values close to zero** produce more regular (hence the name of the techniques) and **stable output**, often leading to better generalisation.

The intuition behind these techniques is that they **constrain** (regularise) or **shrink the coefficient estimates towards zero**, thus **discouraging** learning **more complex** or flexible models, which are indeed the main cause of overfitting.

Mathematically, regularisation consists in **adding a regularisation term to the loss**:

$$J_{\text{TOT}} = J_{\text{LOSS}} + J_{\text{REG}}$$

REGULARISATION TECHNIQUES

L1 regularisation adds a term that depends on the *absolute values of the weights*:

$$J_{\text{L1}} = \lambda \sum_i |w_i|$$

L2 regularisation adds a term that depends on the *sum of the squares of the*

weights:

$$J_{L2} = \frac{\lambda}{2} \sum_i w_i^2$$

In both cases the λ parameter **adjusts the strength** of the regularisation.

🔗 OBSERVATION: WEIGHT DECAY

L1 can have a **greater sparsifying effect** (that is, bringing the weights down to 0) compared to L2.

Remember that weights are *updated proportionally to the gradient* of J . Now that we have this extra regularisation term, it adds a contribution given by:

$$\frac{\partial J_{L1}}{\partial w_i} = \lambda \cdot \text{sgn}(w_i)$$

$$\frac{\partial J_{L2}}{\partial w_i} = \lambda w_i$$

L1 always adds the same contribution, no matter the value of w_i . On the contrary, **L2 contributions become progressively smaller** in value as the weight w_i approaches zero.

Thus, **L1 brings weights down to zero faster**.